

SSP-MMC: алгоритм стохастического кратчайшего пути для оптимизации интервальных повторений

Разбор статьи KDD '22 на русском с комментариями для проекта
Poker_Trainer

Авторы оригинала: Junyao Ye, Jingyong Su, Yilong Cao (MaiMemo, HIT Shenzhen). Разбо

Май 2026

Содержание

Метаданные статьи	1
Краткая суть в трёх абзацах	2
1. Введение: что не так с SM-2 и аналогами	2
2. Датасет MaiMemo	3
3. Кривая забывания и эффект интервала: что в данных	4
4. Модель DHP	4
5. Постановка как Stochastic Shortest Path	5
6. Эксперимент 1: DHP vs HLR на качестве предсказания	7
7. Эксперимент 2: SSP-MMC vs бейзлайны	7
8. Архитектура внедрения в продакшн (Appendix A)	8
9. Что из этого можно взять в Poker_Trainer	8
10. Ограничения статьи и наши отличия	9
11. Если коротко — для разработчика Learn Mode	10
12. Дополнительные источники	10

Метаданные статьи

Название (ориг.): *A Stochastic Shortest Path Algorithm for Optimizing Spaced Repetition Scheduling*

Авторы: Junyao Ye, Jingyong Su (Harbin Institute of Technology, Shenzhen), Yilong Cao (MaiMemo Inc., Qingyuan).

Где опубликовано: ACM SIGKDD '22 (Knowledge Discovery and Data Mining), Вашингтон, 14-18 августа 2022.

DOI: 10.1145/3534678.3539081

Открытый код и данные: <https://github.com/maimemo/SSP-MMC>

Контекст для нас. Это та самая статья, из которой выросло семейство алгоритмов **FSRS** (Free Spaced Repetition Scheduler) — современная замена SM-2 в Anki, RemNote и подобных системах. Понимать, что в ней предлагается, прямо относится к нашему Learn Mode в Poker_Trainer: текущая реализация в `srs.py` написана по SM-2 (1990 г.), а в статье обсуждается принципиально другой подход — основанный на эмпирической модели памяти и оптимальном управлении.

Краткая суть в трёх абзацах

Авторы собрали 220 миллионов событий повторения слов у 828 тысяч пользователей приложения MaiMemo (китайское приложение для изучения английских слов) и впервые в литературе сохранили **временные ряды** этих событий: для каждого слова — последовательность интервалов между повторами и последовательность результатов «помнил / забыл». Они показали, что SM-2-подобные алгоритмы и популярные модели типа HLR (Half-Life Regression) теряют важную информацию, потому что моделируют забывание как функцию только числа повторений или последнего интервала, без учёта истории.

На этих данных авторы построили модель памяти **DHP** (*Difficulty-Halflife-P(recall)*): три переменные состояния (трудность материала, период полураспада памяти, вероятность вспомнить сейчас), три уравнения перехода. Модель сохраняет Марковское свойство (следующее состояние зависит только от текущего), что делает её пригодной для математической оптимизации. По точности предсказания DHP бьёт HLR в 3-4 раза по MAE.

Дальше задачу подбора интервалов формализовали как **Stochastic Shortest Path** на дискретизированном пространстве состояний и решили динамическим программированием (уравнение Беллмана). Цель: довести каждую карточку до целевого периода полураспада с **минимальной стоимостью повторений**. Получили алгоритм **SSP-MMC** (Stochastic Shortest Path – Minimize Memorization Cost). На симуляциях он экономит $\approx 12.6\%$ времени по сравнению с лучшим бейзлайном (THRESHOLD из SuperMemo) и обгоняет ANKI (SM-2), MEMORIZE, HALF-LIFE и RANDOM на всех трёх метриках (THR, SRP, WTL).

1. Введение: что не так с SM-2 и аналогами

Эббингхаус в 1885-м заметил, что забывание подчиняется кривой, а число повторений и интервалы между ними сильно влияют на запоминание. На этом основан принцип интервальных повторений (spacing effect). Все массовые SRS-движки — Leitner Box (1972), SuperMemo SM-2 (1990), Anki — это **набор эвристик с захардкоженными константами**. Авторы пишут (цитата):

“Due to the hard-coded parameters and the lack of theoretical proof, these algorithms cannot adjust to different learners and materials. Meanwhile, their performance cannot be guaranteed.”

То есть три проблемы:

1. Параметры подобраны «на глаз», не из данных.
2. Алгоритм не адаптируется к конкретному пользователю и конкретному материалу.
3. Нет формальной гарантии оптимальности.

Современные ML-подходы тоже неидеальны:

- **HLR / EFC** — игнорируют интервал между повторениями как фактор силы памяти; считают силу функцией только числа повторов.
- **HLR + нейросеть** — игнорируют временной ряд (порядок событий).
- **RL-подходы** (deep reinforcement learning поверх симулятора памяти) — чёрный ящик, симуляторы слишком упрощены, нет интерпретируемости.

К нашему проекту. Все эти проблемы 1-в-1 относятся и к нашему текущему srs.py. Мы используем SM-2 с константами DEFAULT_EASE = 2.5, MIN_EASE = 1.3, фиксированной graduation ladder $0 \rightarrow 1 \rightarrow 3 \rightarrow 3 \times \text{ease}$. Эти значения — наследие SuperMemo 1990 года, никакая мат. оптимизация под покерные диапазоны их не подтверждает. Это нормально для MVP, но важно понимать: цифры взяты с потолка.

2. Датасет MaiMemo

Авторы собрали месяц логов: 220 млн событий повторения, 828 тыс. пользователей, ≈ 100 тыс. английских слов. Одно событие представлено как кортеж:

$$e_i := (u, w, \Delta t_{1:i-1}, r_{1:i-1}, \Delta t_i, r_i)$$

где u — пользователь, w — слово, $\Delta t_{1:i-1}$ — последовательность интервалов с 1-го по $(i-1)$ -й повтор, $r_{1:i-1}$ — последовательность результатов («помнил» / «забыл»), Δt_i — текущий интервал, $r_i \in \{0, 1\}$ — текущий результат.

Главное отличие от датасета Duolingo (который раньше использовался в литературе): сохранён полный временной ряд. Duolingo считал «вероятность вспомнить» как долю правильных ответов в одной сессии, что некорректно (первое успешное повторение в сессии влияет на последующие).

Чтобы сгладить разреженность по словам, авторы сгруппировали слова в **10 кластеров трудности** $d \in \{1, \dots, 10\}$ по доле вспомненных через день после первого знакомства:

- $d = 1$: «самые лёгкие», доля вспомненных $P(\text{recall}) > 0.85$
- $d = 10$: «самые трудные», $P(\text{recall}) \leq 0.45$
- Остальные — равномерно по 8 бакетам

Аналогия с покером. В нашем случае «слово» = (рука, позиция героя, спот, позиция оппонента). «Трудность» естественно мапится в стратегию: риге-карты (open: 1.0) — лёгкие, mixed карты с близкими частотами (open: 0.55, fold: 0.45) — самые трудные, потому что туда сложнее «попасть»: легко промахнуться, выбрав не то действие, хотя оно тоже технически в стратегии. Сейчас у нас этой шкалы трудности нет вовсе — все карточки стартуют с одинаковым ease 2.5.

3. Кривая забывания и эффект интервала: что в данных

На сгруппированных данных авторы строят кривые забывания. Доля вспомненных $\hat{p}_i$ из группы N людей при интервале Δt_i аппроксимируется экспоненциальной функцией:

$$\hat{p}_i = 2^{-\Delta t_i / h_i}$$

где h_i — **период полураспада памяти** (через сколько дней вероятность вспомнить = 50 %).

Ключевое эмпирическое наблюдение из их Figure 3: при одинаковом числе повторов, но разных интервалах между ними, итоговый h существенно отличается — длиннее интервал перед успешным повтором, больше прирост h после. Это и есть «spacing effect» в количественной форме.

4. Модель DHP

Авторы намеренно отказываются от RNN / LSTM, чтобы сохранить интерпретируемость, и строят модель с Марковским свойством на четырёх переменных состояния:

- h — период полураспада (storage strength памяти)
- p — вероятность вспомнить прямо сейчас (retrieval strength). По соотношению $p = 2^{-\Delta t / h}$ она однозначно определяется парой $(\Delta t, h)$.
- r — результат последнего повторения (0/1)
- d — трудность

Уравнения перехода. После события i (был интервал Δt_i , результат r_i):

$$h_i = \begin{cases} h_{i-1} \cdot (\exp(\theta_1 \cdot x_i) + 1), & r_i = 1 \\ \exp(\theta_2 \cdot x_i), & r_i = 0 \end{cases}$$

$$d_i = \begin{cases} d_{i-1}, & r_i = 1 \\ d_{i-1} + \theta_3, & r_i = 0 \end{cases}$$

где $x_i = [\log d_{i-1}, \log h_{i-1}, \log(1 - p_i)]$ — вектор признаков, а $\theta_1, \theta_2, \theta_3$ — обучаемые параметры.

Стартовое значение: $h_1 = -1/\log_2(0.925 - 0.05 \cdot d_0)$ (выводится из группировки трудности).

После фитирования на данных авторы получают конкретные веса. После успешного повтора:

$$h_{i+1} = h_i \cdot (\exp(3.81) \cdot d_i^{-0.534} \cdot h_i^{-0.127} \cdot (1 - p_i)^{0.970} + 1)$$

После забывания:

$$h_{i+1} = \exp(-0.041) \cdot d_i^{-0.041} \cdot h_i^{0.377} \cdot (1 - p_i)^{-0.227}$$

Что из этих формул видно содержательно (важные интерпретации авторов):

- $(1 - p_i)^{0.970}$ в первой формуле — положительная степень: чем ниже p_i к моменту повтора (т. е. чем больше интервал), тем сильнее прирост h . Это и есть **spacing effect** в коэффициентах.
- $h_i^{-0.127}$: чем длиннее уже было h , тем меньше его прирост — **marginal effect**, насыщение.
- $d_i^{-0.534}$: чем сложнее материал, тем меньше прирост за один успешный повтор — **сложные карточки растут медленнее**.
- Во второй формуле (после забывания) $h_i^{0.377}$ — положительная зависимость: даже после провала память не «сбрасывается в 0», у длинной памяти больше остаточная сила.

Сравнение с нашим SM-2. Наш `srs.update_card` меняет интервал по фиксированной лестнице $0 \rightarrow 1 \rightarrow 3 \rightarrow 3 \times \text{ease}$, а `ease` двигается на ± 0.15 или ± 0.20 . Здесь же интервал считается через *предсказанное* h , которое зависит от *всей предыстории карточки*. Это не «магия машинного обучения» — это просто аппроксимация эмпирической кривой забывания тремя интерпретируемыми параметрами. Можно реализовать без нейросетей.

5. Постановка как Stochastic Shortest Path

Сформулируем задачу планировщика повторений в общих терминах. Дано: модель DHP, целевой период полураспада h_N (после достижения карточка считается «выученной»), стоимость одного повтора g (можно различать стоимости успешного и неуспешного: $g(r_k) = ar_k + b(1 - r_k)$).

На каждом шаге, находясь в состоянии (h_k, d_k) , планировщик выбирает интервал Δt_k из допустимого набора $\Delta T(h_k, d_k)$. Результат r_k случаен (Бернулли с вероятностью p_k). Полная стоимость до достижения h_N — случайная величина (потому что число неудач — случайно). Это классический стохастический кратчайший путь.

Уравнение Беллмана (SSP-ММС):

$$J^*(h_k, d_k) = \min_{\Delta t_k \in \Delta T(h_k, d_k)} \mathbb{E}[g(r_k) + J^*(f(h_k, d_k, \Delta t_k, r_k))]$$

В лоб: пробуем все возможные интервалы, для каждого считаем матожидание (по двум исходам с весами p_k и $1 - p_k$) суммы текущей стоимости и ожидаемой остаточной стоимости из следующего состояния. Выбираем минимум.

Чтобы это можно было считать на компьютере, h дискретизируется логарифмически:

$$h_{\text{index}} = \lfloor \log(h) / \log(\text{base}) \rfloor$$

(потому что в наших уравнениях h растёт примерно экспоненциально, и при равномерной сетке высокие h были бы слишком разреженными).

Итеративный алгоритм. Заводим матрицу стоимости $J[d_N][h_N]$, инициализируем ∞ . В терминальном состоянии $h = h_N$ стоимость = 0. От верхней границы трудности вниз обновляем по уравнению Беллмана до сходимости. Параллельно сохраняем матрицу оптимальной стратегии π^* — какой интервал Δt выбирать в каждой ячейке. Псевдокод (по сути их Алгоритм 1):

вход: a, b, d_N, h_N

выход: $\pi^*[d_N][h_N], J[d_N][h_N]$

для d от d_N до d_0 :

$J[d][h_0..h_{N-1}] = \infty$

$J[d][h_N] = 0$

пока изменение $J[d][h_0] > 0.1$:

$J_{\text{prev}} = J[d][h_0]$

для h от h_{N-1} до h_0 :

для каждого Δt из $\Delta T(d, h)$:

$p = 2^{-\Delta t/h}$

$h_{r1} = f(h, p, d, r=1)$

$h_{r0} = f(h, p, d, r=0)$

$J_{\text{new}} = p \cdot (a + J[d][h_{r1}]) + (1-p) \cdot (b + J[d+2][h_{r0}])$

если $J_{\text{new}} < J[d][h]$:

$J[d][h] = J_{\text{new}}$

$\pi^*[d][h] = \Delta t$

$\Delta J = J_{\text{prev}} - J[d][h_0]$

Сходится за конечное число итераций (классический результат теории SSP с положительными стоимостями и существованием терминала).

Прикинем масштаб для нас. Если взять $h_N = 60$ дней (а не годы, как у них), $d \in \{1..10\}$, дискретизация h — пусть 30 бинов, набор кандидатов Δt — 30 значений (от 1 дня до h_N), это $10 \times 30 \times 30 = 9000$ обновлений на итерацию, и сходится за десятки итераций. Считается за секунды на ноутбуке. Никакой GPU не нужен. Это абсолютно посылно для одного скрипта в проекте, например `src_ssp.py`.

6. Эксперимент 1: DHP vs HLR на качестве предсказания

Авторы сравнивают свою модель DHP с HLR (Half-Life Regression, Duolingo 2016) по MAE и MAPE на предсказании следующего полураспада. Результаты (Таблица 3 оригинала):

Метрика	Recall — MAE	Recall — MAPE	Forget — MAE	Forget — MAPE
DHP	11.93	26.16 %	0.44	31.99 %
HLR	41.70	45.75 %	7.08	112.7 %

DHP бьёт HLR в 3-4 раза по абсолютной ошибке, и **особенно сильно — на исходе «забыл»** (16-кратная разница по MAE). HLR не различает истории $r_{\{1:2\}} = 1,0$ и $r_{\{1:2\}} = 0,1$ (помнил-забыл vs забыл-помнил), а DHP различает, потому что у него временной ряд проброшен в признаки x_i .

7. Эксперимент 2: SSP-MMC vs бейзлайны

Симуляционная среда: модель DHP как симулятор пользователя; бюджет 600 секунд (10 минут) в день, стоимость успешного повтора 3 с, провального — 9 с, целевой период полураспада $h_N = 360$ дней, симуляция 1000 дней.

Бейзлайны:

- **RANDOM** — интервал случайный из $[1, h_N]$.
- **ANKI** — наш старый знакомый, вариант SM-2.
- **HALF-LIFE** — интервал = текущий h (то есть до момента, когда $p = 0.5$).
- **THRESHOLD** — повторять, когда $p \leq 0.90$ (дефолт SuperMemo).
- **MEMORIZE** — алгоритм оптимального управления Tabibian et al. 2019 (точечные процессы).

Метрики:

- **THR** — сколько слов достигло целевого h_N (главная метрика, совпадает с оптимизируемой целью SSP-MMC).
- **SRP** — суммарная вероятность вспомнить.
- **WTL** — сколько слов всего «учил».

Результаты (день, на который THR достиг 6000 слов — отметка ★ на Figure 10):

Алгоритм	Дней до 6000 выученных
SSP-MMC	466
THRESHOLD	533
ANKI (SM-2)	569
MEMORIZE	793
RANDOM	— (не дошёл)

Преимущество SSP-MMC над THRESHOLD: $(533 - 466)/533 \approx 12.6\%$ экономии времени. Над ANKI — $(569 - 466)/569 \approx 18.1\%$. Над MEMORIZE — больше 41 %.

Любопытный нюанс по WTL (число «начатых» слов): на ранних днях RANDOM лидирует, потому что он реже назначает повторения, и пользователь успевает познакомиться с большим числом новых слов. Но это иллюзия — он их забывает почти все. На длинной дистанции и по THR, и по SRP RANDOM проигрывает.

8. Архитектура внедрения в продакшн (Appendix A)

Приложение MaiMemo использует двухслойную архитектуру:

Local (на устройстве):

- Local scheduler — назначает следующее повторение по последней актуальной policy.
- Local memory state predictor — пересчитывает (h, d) после каждого ответа.
- User review logs — пишутся локально, заливаются в облако раз в сутки.

Remote (в облаке):

- Full review logs — streaming-загрузка событий в data warehouse.
- Data pre-process ETL — периодически считывает h и d для всех слов.
- DHP memory model — фитируется на свежих данных, выдаёт обновлённые θ .
- SSP-MMC scheduling algorithm — на новых θ заново строит матрицу π^* .
- Конфиг с θ и π^* пушится на устройства.

Холодный старт. Когда данных по пользователю ещё нет, работает SM-2; параллельно собираются логи. Как только наберётся достаточно событий, fit-ятся параметры DHP, считается π^* , и SM-2 заменяется на оптимальную policy. Цикл повторяется итеративно.

Для нас это означает следующее. У нас один пользователь и один компьютер — нам не нужна вся эта инфраструктура. Достаточно одного скрипта `srs_ssr.py`, который раз в сутки/неделю поджигает накопленные `srs_state/*.srs.json`, фитит DHP под наши покерные карточки, считывает π^* и сохраняет матрицу policy в JSON. Run-time планировщик читает эту матрицу и для текущего состояния карточки (h, d) выдаёт ближайший интервал.

9. Что из этого можно взять в Poker_Trainer

Несколько направлений, в порядке возрастания сложности:

1. (Дешёвое улучшение) Подкрутить SM-2-константы по своим логам. Прямо сейчас у нас фиксированные `DEFAULT_EASE = 2.5`, `MIN_EASE = 1.3`, `graduation 0→1→3`. Достаточно собрать пару недель статистики попаданий по карточкам, посмотреть фактическую кривую забывания на покерных диапазонах, и подкрутить эти константы под наш материал. Это не FSRS, но это первый шаг.

2. (Средняя стоимость) Ввести понятие трудности карточки d . Сейчас у нас неявно вся информация о трудности замешана в `ease_factor`. Это смешивает

«материал трудный» и «я долго не мог». Авторы статьи разделяют: d — свойство материала, h — свойство индивидуальной памяти на эту карточку. Для нас естественно положить d через свойства спота:

- чистый pure-RFI (`{open: 1.0}`) $\rightarrow d = 1$ (легче не бывает)
- близкие mixed (`{open: 0.55, fold: 0.45}`) $\rightarrow d = 10$ (легко промахнуться по ощущению, хоть оба ответа GOOD)
- средний микс (`{3bet: 0.7, call: 0.3}`) $\rightarrow d \approx 5$

И дальше учитывать d при увеличении/уменьшении интервала.

3. (Дороже) Заменить SM-2 на минимальный DHP + SSP-MMC. Реализовать модель DHP как чистую Python-функцию `predict_h(h_prev, d, p, r) -> h_new` с тремя массивами параметров $\theta_1, \theta_2, \theta_3$. Стартовать с параметров, взятых из статьи (формулы 12 и 13 выше). Реализовать SSP-MMC как одноразовый скрипт `precompute_policy.py`. Сохранять policy в JSON. В run-time использовать matrix-lookup вместо SM-2-формул.

Главное — у нас уже есть всё, что нужно для логирования: `Card.total_seen`, `Card.total_correct`, `Card.consecutive_correct`, `last_seen`, `next_review`, и история ответов в `history.json` для Practice Mode. Останется только добавить интервал между повторами и не выбрасывать его в `srs_state`.

4. (Дорого, но интересно) Дообучать θ на своих логах. Когда накопится несколько тысяч событий, провести fit DHP под себя и заменить параметры. Тут уже понадобится оптимизатор (`scipy.optimize` или PyTorch минимально) и аккуратный train/val split — но это абсолютно посильно для одного пользователя и одного материала.

10. Ограничения статьи и наши отличия

Что в статье не идеально:

- Симуляция гоняется в среде, обученной той же моделью DHP. Это самосогласованный тест: алгоритм может быть оптимален для DHP-симулятора, но это не доказывает оптимальность на живых людях. Авторы это упоминают.
- Параметры подбирались на одном типе материала (английские слова). Для другого материала (математические формулы, музыкальные пьесы, **покерные диапазоны**) формы кривой могут отличаться.
- Не учитывается «семантическая близость» карточек (запутать AKs vs UTG и AKs vs MP — типичная человеческая ошибка, которую DHP не моделирует).

Чем наш случай отличается от их MaiMemo:

- У нас один пользователь, у них миллионы. Авторам нужна стабильность параметров для большого населения, нам — адаптация под себя.
- У нас стратегия может быть mixed (`open: 0.7, fold: 0.3`), у них слово либо вспомнил, либо нет. Мы это уже обходим в `grade_answer` (оба in-strategy действия = GOOD), и эта логика поверх FSRS-подобного движка тоже работает.
- У нас целевой h_N может быть меньше: для покерных диапазонов «помнить год» — это слишком, достаточно «помнить пока я играю на этих лимитах», что-то вроде 30-90 дней.

- Стоимости a и b у нас семантически другие. У них $a = 3$ с, $b = 9$ с (физическое время на флэшкарту). У нас «стоимость» — это и время, и риск конкретной ошибки за реальной игрой (за неправильный fold AA пре получишь больше, чем за неправильный fold 22 в плохом споте). В принципе g можно сделать функцией спота: BB-cost на префлопе известно, выкатить веса можно эмпирически.
-

11. Если коротко — для разработчика Learn Mode

- Текущий SM-2 — рабочий, но это карикатура на современный SRS. Авторы статьи показали +12.6 % к ANKI на простом материале. На нашем материале (мало карточек, mixed-стратегии) разрыв, скорее всего, ещё больше.
 - Минимальный апгрейд: добавить переменную `difficulty` в Card, инициализировать её по типу спота (pure / mixed / near-pure), и хотя бы простую формулу прироста h от трудности и от p к моменту повтора (формулы 12/13 выше — рабочие стартовые веса).
 - Полный апгрейд: реализовать SSP-MMC оффлайн-скриптом, дальше run-time только читает матрицу.
 - Это всё **не требует ни нейросетей, ни ML-фреймворков**. Чистый Python + numpy. Объем кода — сопоставим с текущим `srs.py`.
 - Не торопись. Frontend Learn Mode — приоритет №1; FSRS-апгрейд — это уже после того, как накопится статистика твоих ответов.
-

12. Дополнительные источники

- Оригинал статьи: ACM DL, DOI 10.1145/3534678.3539081.
 - Код и датасет: <https://github.com/maimemo/SSP-MMC>.
 - FSRS (open-source продолжение): <https://github.com/open-spaced-repetition/free-spaced>
Реализации на Python и Rust, документация, бенчмарки. Это то, во что превратился SSP-MMC после двух лет развития сообществом.
 - Обсуждение в Anki-сообществе: ключевые слова «FSRS-4.5», «FSRS-5» — там есть готовый набор параметров и описание изменений относительно статьи 2022.
-

Этот документ — краткий конспект и комментарий для личного использования по статье Ye, Su, Cao (2022). Все формулы и численные результаты — из оригинальной статьи. Все интерпретации и проекции на проект `Poker_Trainer` — собственные, оригинальной статье не принадлежат.